

PTC Sunum Rehberi — baştan sona

Tarih: 2026-09-08 · **Slayt:** 11 sayfa · **Süre:** 30 dk anlatım + 10 dk soru

Bu dosya sunumu yönetmek için. Slayt sırasına göre: her sayfada **ne diyeceğin, hangi sayıyı söyleyeceğin, neyi söylemeyeceğin**.

- Slayt (canlı):** <https://claude.ai/code/artifact/ba545b49-2790-4584-b2e0-aae256a1e12a>
- Slayt (yerel, internetsiz de açılır):** PTC_Hizli_Tur.html
- Ölçüm raporu:** PTC_Hata_Bicimleri_Kiyas.md

Sunum sırasında yalnızca §8'i aç — sayfa sayfa, baştan sona okunabilir konuşma metni.

0 · Sunumdan 15 dakika önce

Aç

```
cd ~/Desktop/Data-Science-Reports/ptc_art
S=/tmp/ptc-log; mkdir -p $S

nohup kubectrl port-forward svc/artifact-service 8080:8080 > $S/pf.log 2>&1 & disown
PYTHONPATH=src nohup ../ptc_sec/.venv/bin/uvicorn \
  grounded_assistant.web.app:app --port 8123 --log-level info > $S/web.log 2>&1 & disown
nohup cilium hubble port-forward > $S/hubble.log 2>&1 & disown
nohup kubectrl port-forward -n kube-system svc/hubble-ui 12000:80 > $S/hubble-ui.log 2>&1 & disown
```

Doğrula

```
kubectrl get pods # artifact-service · minio · tool-gateway Running
curl -s -o /dev/null -w "%{http_code}\n" http://localhost:8123/konsol # 200
hubble status | grep Healthcheck # Ok
```

Kısayolu tanımla

```
bak() { /home/altan/Desktop/Data-Science-Reports/ptc_sec/.venv/bin/python \
/home/altan/Desktop/Data-Science-Reports/ptc_art/scripts/artifact_bak.py "$@"; }
```

Ekranda hazır dursun

Sekme	Ne
1	PTC_Hizli_Tur.html — slayt
2	http://localhost:8123/konsol — PoC
3	http://localhost:12000 — Hubble UI
4	Terminal — hubble observe --label app=ptc-sandbox -f hazır, ENTER'a basılmamış

Slayt kontrolleri

→ ← sayfa o genel bakış (11 sayfa ızgara) Esc kapat

Depoyu **önceden doldur**: Hatlar sekmesinden PL-A'yı bir kez çalıştır. Demoda boş depo göstermek istemezsin. Alias demosu için depoda üç rapor.json sürümü prop olarak duruyor (bkz. canlı demo ④).

1 · Anlatım planı — 11 sayfa

Aşağıdaki süreler **33 dakikalık** bir plan. 30'a sığdırman gerekirse 6 ve 7'den kıs; 2, 3, 5, 9, 11'e dokunma.

#	Sayfa	dk	Tek cümlelik amaç
1	Sandbox ölür, ürettiği kalır	2	Problemi kur
2	Beş soru ★	4	Haritayı göster — sunumun zemini
3	Dört aile ★	4	Tez burada
4	Dört ürün, dört akış	3	Tezi örnekle
5	Red Hat / OpenShift AI ★	4	En yakın komşu, ve neden yetmiyor
6	Tekton · Argo · Databricks · Devin	2	Kalanları kapat
7	Keşif ve kayıt defteri	2	İkinci sorun: bulmak
8	Çağrı değil, beyan	3	Bizim cevabımız
9	İki container + OpenShift ★	4	Ölçüm ve hedef ortam
10	Kod patlıyor, ne dönüyor	2	İkinci eksen
11	Ölçülmüş karşılaştırma ★	3	Kanıt

Her sayfanın sonunda bir **geçiş cümlesi** var. Sunumu ayakta tutan şey sayfalar değil, aralarındaki bağlantı — o cümleleri atlama.

Sayfa 1 — Sandbox ölür. Ürettiği kalır.

Aç:

"Kod çalıştıran her sistemin aynı çelişkisi var. Sandbox yaşarsa bir çalıştırma diğerine bulaşır — izolasyon yok. Sandbox ölürse 40 saniyelik iş her turda tekrarlanır."

Çizimi göster: kesikli kutu sandbox, ok, silindir depo.

"Çözüm ikisinin ortasında değil: **sandbox ölecek, ürettiği kalacak**. Ama bir şart var — bizim sandbox'ımızdaki kodu **LLM yazıyor**. Yani güvenilmez. Bu, sunumun geri kalanının tek dayanağı."

Söyleme: Henüz çözümünden, mimariden, MinIO'dan bahsetme. Bu sayfa sadece problemi kuruyor.

Geçiş:

"Bu problemi ilk biz görmedik. Önce piyasanın nasıl çözdüğüne bakalım."

Sayfa 2 — Herkes aynı beş soruyu cevaplamak zorunda ★

Bu sayfa sunumun zemini. Buradaki örüntü tutmazsa, 3. sayfadaki tez havada kalır. Dört beat hâlinde anlat; tabloyu **asla baştan sona okuma**.

Beat 1 — bu tablo neden var (30 sn)

"On ürüne baktım: Anthropic, OpenAI, Cloudflare, Google, Red Hat, Argo, Databricks, AWS, Microsoft, E2B ve Devin. Hepsi farklı bir problem çözdüğünü sanıyor — biri sandbox satıyor, biri pipeline, biri ajan framework'ü. Ama hepsi **aynı beş soruyu** cevaplamak zorunda kalmış. Bu tablo o beş sorunun cevapları."

Beat 2 — beş soru (45 sn)

Ekrana bakmadan say, sonra tabloya dön:

S1	NEREDE	kod nerede koşuyor?
S2	ÖMÜR	ne kadar yaşıyor?
S3	VERİ	nasıl girip çıkıyor?
S4	ANAHTAR	depo anahtarı nerede?
S5	DEFTER	künye tutuluyor mu?

"Slaytta dördü sütun olarak duruyor. S1 ve S3'ü 'Tezi' sütununa katladım — bir ürünün nerede çalıştığı ile veriyi nasıl taşıdığı, aynı tasarım kararının iki yüzü."

"**Hani beş soru?**" diye sorulursa cevabın bu. Sormadan da söyleyebilirsin, tabloya bakan biri saymaya kalkıyor.

Beat 3 — dört satırı konuş, gerisini konuşma (90 sn)

On satırın hepsini okursan kimse hiçbirini hatırlamaz. **Dördünü seç**, sırayla:

① Anthropic — "container'ı sakla"

"Sorunu kalıcılık değil, **ortamı yaşatmak** olarak görüyor. Container 30 gün duruyor, beş dakika hareketsizlikte dondurulup geri yükleniyor. Anahtar sandbox'ta yok. Defteri var — file_id. Ama parantez içine bakın: **soy yok**. file_abc123 size dosyayı verir, neyden türediğini söylemez."

② Cloudflare — "kod yaz, tool çağırma"

"Bu ürünün tezi artifact'le ilgili bile değil: *modeller kod yazmayı tool çağırmaktan daha iyi biliyor*. R2 bucket'ını sandbox'a mount ediyor. Defter sütununa dikkat — orada 'yok' yazmıyor, '**olamaz**' yazıyor."

Burada bir saniye dur. Bu hücre sunumun anahtarı.

"Neden olamaz? Çünkü mount'ta araya girecek yer yok. Kod dosyaya yazıyor, bucket'a düşüyor, arada kimse durmuyor. Kimse durmuyorsa künye de tutulamaz."

③ Red Hat / KFP — "DAG yazılıdır"

"Tablodaki en ilginç satır bu. Defteri **en zengin** olan ürün — MLMD: tip, soy, beyan edilen girdiler, hepsi var. Ama anahtar sütununda **büyük harflerle EVET** yazıyor: depo anahtarı sandbox'ın içinde."

"Yani en iyi defteri tutan ürün, aynı zamanda anahtarı kodun yanına koyan ürün. Bu bir çelişki değil — bir **varsayım**. 5. sayfada o varsayımın ne olduğunu göreceğiz."

④ Databricks — "denetimli mount"

"Ve bir istisna: Databricks mount ediyor **ama** defteri var. Nasıl? Aracıyı kaldırmamışlar — **sürücünün içine** gömmüşler. Her erişimde Unity Catalog'a soruluyor."

Kalan altı satır için tek cümle yeter:

"Gerisi iki gruba ayrılıyor: altyapı satanlar — AWS, Microsoft, E2B, Modal, Vercel — 'depoyu sen getir' diyor, defter tutmuyor. Ve Devin, soruyu tamamen ortadan kaldırıyor: sandbox'ı hiç öldürmüyor."

Beat 4 — örüntüyü söyle (45 sn)

Parmağını defter sütununda yukarıdan aşağı gezdir:

"Şimdi tek bir sütuna bakın. Defteri olanlar: Anthropic, Google ADK, Red Hat, Databricks. Olmayanlar: Cloudflare, AWS, E2B, Modal, Vercel. İkinci grubun ortak özelliği ne? **Hepsi mount ediyor.**"

Sonra cümleyi söyle — yavaş:

"Kayıt defteri yalnızca yazma yolu bir bileşenden geçtiğinde ayakta kalıyor."

"Araştırmamanın en keskin bulgusu bu. Ve bu bir tercih meselesi değil: mount edince künye tutmak **imkânsız** hâle geliyor."

Savunma — gelmesi muhtemel iki itiraz

"Argo'da da bileşen var ama defteri yok. Tezin çürüdü."

"Tez tek yöne okunuyor. Mount ederseniz defter **tutamazsınız** — bu kesin. Bileşen koyarsanız **tutabilirsiniz** — ama tutmak zorunda değilsiniz. Argo tutmamayı seçmiş, çünkü orada her çalıştırma kendi klasörüne yazıyor, aynı adın iki sürümü hiç çakışmıyor. Bizde ajan karar verdiği için katalog şart. Yani araya giren katman defterin **ön koşulu**, garantisi değil."

"Databricks mount ediyor ama defteri var, istisna değil mi?"

"Tam tersine, kuralı doğruluyor. Databricks aracıyı **kaldırmamış**, sürücünün içine taşımış. Araya giren biri hâlâ var — sadece görünmüyor."

Söylemeyeceklerin

x tabloyu satır satır okumak	10 satır = 0 akılda kalan
x "hepsi yanlış yapıyor"	hepsi kendi bağlamında doğru
x OpenAI satırını kesin konuşmak	bu turda yeniden doğrulanmadı

Sorulursa iki dürüstlük notu:

OpenAI satırı	bu turda birincil kaynaktan yeniden doğrulanmadı, önceki taramaya dayanıyor — dokümanda işaretli
AWS · Microsoft	iki ürün tek satırda, sıkıştırılmış

Geçiş — bu cümleyi ezberle

"Peki neden mount edince künye tutulamıyor? Cevap, **baytı kimin taşıdığında**. Sonraki sayfa bunun dört yolunu gösteriyor."

Sayfa 3 — Baytı kim taşıyor: dört aile ★

Sunumun tezi burada. Acele etme.

Dört çizimi tek tek göster, her birine bir cümle:

A · MOUNT	kod → FUSE → bucket	araya girecek yer YOK
B · SARMALAYICI	launcher + kod AYNI container	anahtar kodun yanında
C · SINIR	kod → izin → AYRI container	kod anahtarı göremez
D · DENETİMLİ	mount var, sürücü UC'ye soruyor	

"**A**'da kod doğrudan bucket'a yazıyor. FUSE bunu dosya gibi gösteriyor ama altında HTTP var. Araya girecek yer yok — 2. sayfadaki 'olamaz' hücresinin sebebi bu."

"**B**'de bir taşıyıcı var ama kullanıcı koduyla **aynı container'da**. Yani var, ama atlanabilir."

"**C**'de taşıyıcı ayrı bir güven alanında — kod ona ulaşamıyor. Bizim yerleşimimiz bu."

"**D** tek istisna: mount var ama sürücü her erişimde izin soruyor."

Kapanış — Argo'nun kanıtı:

"Bunu biz düşünmedik, Argo deneyerek buldu. Dört farklı yerleşim denemişler — docker, kubelet, k8sapi, pns. v3.4'te **hepsini kaldırmışlar**. docker için gerekçeleri tek cümle: *breaks security completely*."

Bu alıntı sunumun en güçlü kartı — bir tasarım tercihinin başkasının acı deneyimi doğruluyor.

Geçiş:

"Şimdi bu dört aileyi gerçek ürünlerde görelim."

Sayfa 4 — Dört ürün, dört akış

Dört çizimi tek tek gösterme; **ikiye böl**:

"Üstteki ikisinde platform araya giriyor. Anthropic'te kod \$OUTPUT_DIR'a yazıyor, iş bitince **platform o dizine bakıyor**, dosyayı Files API'ye koyuyor ve konuşmaya bir fiş dönüyor: `file_abc123`. Container ölse bile bayt kalıyor."

"Alttaki ikisinde giren yok. Cloudflare mount ediyor — bakacak kimse yok. ADK ise baytı **hiç taşımıyor**: sadece isimleri her turda talimatlara yazıyor, içeriği model isteyince ve **yalnızca o isteğe** ekliyor."

Sayı: Anthropic 30 gün · OpenAI 20 dakika. İkisi de keşif sorununu **sandbox'ı yaşatarak** çözüyor — bizde o kapı kapalı.

Geçiş:

"Bunlar ajan tarafı. Şimdi pipeline tarafına, bize en yakın olana bakalım."

Sayfa 5 — Red Hat / OpenShift AI ★

En yakın komşumuz. İki bölümde anlat.

Önce anatomi

init container	kfp-driver	.uri'leri MLMD'den ÇÖZER
main container	kfp-launcher	PID 1
└ KULLANICI KODU		launcher'ın ALT SÜRECİ

"Kod S3'ü hiç görmüyor. `cikti.path`'e yazıyor — yerel bir yol. Launcher onu `.uri`'ye taşıyor. İndirme kod **başlamadan** bitiyor, yükleme kod **bittikten sonra** başlıyor. Kod hiçbir şey indirmiyor."

Sonra kritik fark

"Ama launcher kullanıcı koduyla **aynı container'da**, ve kodu **alt süreç** olarak çalıştırıyor. Alt süreç ebeveyninin ortamını devralır — aynı env, aynı dosya sistemi, aynı ağ. Yani kod launcher'ı atlayıp doğrudan S3'e konuşabilir."

Sorulursa: ortam temizlense bile `/proc/1/environ` aynı UID'den okunabiliyor. **Aynı container'da sır saklanamaz** — mesele dikkatsizlik değil, süreç modeli.

Hemen ardından — bu cümle olmazsa haksızlık edersin:

"**KFP için bu doğru karar.** Oradaki kodu bir insan yazdı, gözden geçirdi, git'te duruyor. Sarmalayıcının işi güvenlik değil **kolaylık** — bileşen yazarını S3 kodu yazmaktan kurtarıyor. Bizde kodu LLM yazıyor; aynı karar savunulamaz."

Geçiş:

"Kalan dört desen daha var, hızlıca geçeyim."

Sayfa 6 — Tekton · Argo · Databricks · Devin

Hızlı geç, iki şeyi vurgula:

"**Argo bizim yerleşimimizin kaynağı:** argoexec kullanıcının imajından ayrı bir imaj, anahtar orada, kod ona erişemiyor."

"**Devin** ise soruyu ortadan kaldırıyor — sandbox'ı hiç öldürmüyor, microVM snapshot'la RAM'i bile saklıyor. Bedeli süresiz yaşayan bir makine. Bizim sandbox 4 saniye yaşıyor, ikisi de bize kapalı."

Tekton bir cümle: artifact = **referans**, kap değil — *"only metadata and attestations are stored, not artifact content."*

Geçiş:

"Buraya kadar baytı **taşımayı** konuştuk. Bir sorun daha var: onu **bulmak**."

Sayfa 7 — Keşif ve kayıt defteri

"Tek bir şey belirliyor: sandbox yaşıyor mu. Yaşıyorsa 1s yeter, keşif bedava. Pipeline dünyasında ise soru hiç doğmuyor — DAG'ı insan yazıyor, girdi bağlanmış geliyor."

"**Bizde iki şart birden var:** ajan karar veriyor **ve** sandbox ölüyor. Bu ikisi bir arada olan başka bir yer bulamadım."

Sağdaki tablo için tek cümle:

"Aynı addan 12 sürüm birikince kim ne yapıyor? MLflow alias koyuyor, KFP'de çakışma imkânsız çünkü yol run-id içeriyor. Biz alias'ı MLflow'dan aldık."

Geçiş:

"Piyasa bitti. Şimdi biz ne yaptık."

Sayfa 8 — Çağrı değil. Beyan.

"Eskiden kod çalışırken `load_artifact()` diye bir ağ çağrısı yapıyordu. Şimdi sadece **beyan** ediyor: `inputs=["ozet.json"]`. Dosya kod başlamadan yerine konuyor, kod düz `open()` yazıyor."

Üç biçim:

<code>ozet.json</code>	→ <code>/output/ozet.json</code>	kendi
<code>wf-abc123/ozet.json</code>	→ <code>/artifacts/wf-abc123/...</code>	ÇAPRAZ
<code>rapor.pdf@onaylanmis</code>	→ <code>/artifacts/_alias/rapor.pdf</code>	SABİT

"KFP ve Argo tam böyle yapıyor. Tek fark: onlarda beyanı **insan** yazıyor, YAML'da. Bizde **model**."

Bu sayıyı mutlaka söyle:

100 dosya ürettik, kod bir tanesini okudu

beyansız	→	100 dosya	iner	·	soy	100	EBEVEYN
beyanlı	→	2 dosya	iner	·	soy	2	ebeveyn

"Asıl bedel indirme değil, **soy şişmesi**. 100 ebeveynli bir artifact'in soyu hiçbir şey anlatmıyor. Beyan tam bunun için var — ve soyu da beyandan çıkarıyoruz, MLMD'nin `DECLARED_INPUT` olayı gibi."

Geçiş:

"Peki bu nerede çalışıyor?"

Sayfa 9 — İki container, bir /output ★

"Pod'da iki container var. Sidecar'da S3 anahtarı ve kapsam jetonu. Sandbox'ta LLM'in kodu ve **hiçbir sır**. Aralarında sadece `/output` paylaşılıyor. Container'lar ortam paylaşmıyor — 5. sayfadaki alt süreç problemi burada yok."

Ölçüm — okuyarak göster:

S3 kimlik bilgisi	[]
S3 SDK kurulu	hayır
MinIO'ya IP ile	TimeoutError
internet	ConnectionError

sandbox'ın ağ çağrısı: 0

OpenShift kısmı — sorulmadan söyle:

"restricted-v2 SCC ile tam akışı koşturduk: rastgele UID, `runAsNonRoot`, tüm capability'ler düşürülmüş. **SCC'nin dayattığı hiçbir kısıt bizi kırmıyor**. Kalıcılık için gereken tek şey bir S3-uyumlu depo: endpoint, bucket, anahtar. Kod hem OBC hem OpenShift AI connection sözleşmesini okuyor — geçiş bir yapılandırma değişikliği, kod değişikliği değil."

Geçiş:

"Buraya kadar artifact'i konuştuk. Bir eksen daha var: kod patladığında ne oluyor."

Sayfa 10 — Kod patlıyor. Modele ne dönüyor?

Altı kutuyu tek tek okuma, örüntüyü söyle:

"Üçünde aynı üçlü var: **çıkış kodu, tam traceback, hata anında korunan stdout**. Bizim eski hâlimiz üçünü de kaybediyordu — üstelik metin modele *dur* diyordu: 'Tahmini bir değer üretme.'"

"smolagents bilinçli bir takas yapıyor: traceback yerine **patlayan satırın kaynak metnini** veriyor."

Geçiş:

"Bunu iddia olarak bırakmadık — ölçtük."

Sayfa 11 — Aynı hata, on biçim: ölçülmüş ★

"On biçimlendiriciyi kurup aynı beş arızaya soktuk, yedi ölçütte puanladık. Kendi biçimimiz **gerçek kod** — `entrypoint.py` içe aktarılıp çağrılıyor, kopya değil. Diğer dokuzu belgelenmiş davranıştan yeniden kuruldu."

Üç bulgu:

1. **Kırpma yönü hatayı yiyor.** SWE-agent'in varsayılan şablonu baştan

kesiyor; hata en sonda olduğu için 160 KB'lık çıktıda `ValueError` modele hiç ulaşmıyor. `bash_only` yapılandırması baş-yarı + son-yarı alıyor, kaybetmiyor. **23 → 26**. Aynı ürün, iki şablon.

1. **Kırpmanın bedeli.** Tek bir hatada AutoGen 211 360, OpenHands 211 328,

Anthropic 215 415 bayt. Bizim 20 615, Codex 10 113.

1. **Talimat sütunu neredeyse boş.** On biçimden dokuzu ne olduğunu anlatıyor,

ne yapılacağını söylemiyor.

Ve dürüstlük cümlesi — atlarsan sorarlar:

"Ölçüt listesi bizim araştırmamızdan çıktı ve bizim biçimimizi de o araştırma şekillendirdi. Sıralamada **dairesellik var**. Kıyaslanabilir olan sinyal değil, **sinyal/bayt**: Codex aynı 30/35'i 2 356 baytla taşıyor, biz 4 538'le. Ve bizde eksik olan bir şey var — **tekrar tespiti**, o sadece OpenHands'te."

Bu kapanış sunumu güçlendiriyor: kendi açığını kendin söylüyorsun.

2 · Canlı demo (5 dk, isteğe bağlı)

Slayt 9'dan sonra yap. Üç adım, hepsi bugün doğrulandı.

① Terminalde Hubble'ı başlat

```
hubble observe --label app=ptc-sandbox -f
```


② Konsolda PL-A'yı çalıştır

Hatlar → **Ticket İşleme Hattı** → *Hattı çalıştır*. Dört adım, dördü de ayrı pod. Panelde adımlar sırayla yeşile döner.

"Her adım ayrı bir Kubernetes Job'u. query ve alias adımları **pod açmaz** — keşif sandbox'ta değil, host tarafında."

③ Ürettiğinin içini göster

Kısayolu **sunumdan önce** tanımla (\$0'da da var):

```
bak() { /home/altan/Desktop/Data-Science-Reports/ptc_sec/.venv/bin/python \  
        /home/altan/Desktop/Data-Science-Reports/ptc_art/scripts/artifact_bak.py "$@"; }
```

```
bak --liste  
bak processed-result.json
```

Künye + soy + gerçek tablo çıkar. Hangi dizinden çağırdığın fark etmiyor.

④ Alias — istersen (2 dk)

En çok soru gelen yer burası; canlı göstermek en kısa yolu.

```
bak rapor.json          # → sürüm 3   "en yeni sessizce kazandı"  
bak rapor.json@onaylanmis # → sürüm 1   "etiketi ben koydum"  
bak art_409b            # sürümün kendisi hiç değişmedi
```

"Alias taşınır, artifact değişmez — git'teki tag ile commit gibi. Ve alias kendiliğinden kazanmaz: `ad@alias` diye **adılla** istemen gerekiyor."

Depoda üç `rapor.json` sürümü prop olarak duruyor. Etiket takılı değilse:

```
T=$(PYTHONPATH=src \  
  ../ptc_sec/.venv/bin/python -c "  
from dotenv import load_dotenv; load_dotenv()  
from grounded_assistant.agent.graph import _kapsam_jetonu  
print(_kapsam_jetonu('demo-alias'))")  
  
curl -s -X PUT -H "X-Scope-Token: $T" \  
  "http://127.0.0.1:8080/artifacts/art_409b34e062da/alias?alias=onaylanmis"
```

Demo çökerse

Panik yok, cümle hazır:

"Ortam sorunu — ölçümler zaten slaytta, hepsi bu cluster'dan alındı."

Sonra sayfa 9'daki ölçüm bloğunu göster ve devam et.

3 · Soru-cevap hazırlığı

"Files API tam olarak ne?"

"Anthropic'in dosya dolabı. Kod \$OUTPUT_DIR'a yazıyor, **platform o dizine bakıyor**, dosyayı kaydediyor ve konuşmaya bir file_id dönüyor. Container ölse bile bayt orada kalıyor."

Takip gelirse: **fış var, soy yok**. file_abc123 neyden türediğini söylemiyor. Sürüm, alias, arama da yok. İki ömrü karıştırma: **container 30 gün, Files API dosyası silinene kadar**.

"Bucket, S3, MinIO — farkları ne?"

tablo	≈	bucket	KAVRAM
SQL	≈	S3 API	PROTOKOL
PostgreSQL	≈	MinIO	ÜRÜN

"Bucket bir ürün değil, kavram. S3 bir ürün değil, protokol. MinIO ürün."

"Neden MinIO?"

"ODF yoktu. Ama bu bir icat değil: **OpenShift'in kendisinde nesne deposu yok** — depolama dokümanı baştan sona blok ve dosya. Red Hat'in kendi AI ürünü bile pipeline server için '*S3-compatible object storage bucket*' şart koyuyor, veritabanı için varsayılan sunarken depolama için sunmuyor. Seçim bizi bağlamıyor: S3 API ortak olduğu için ODF'ye ya da kurumsal S3'e geçiş bir endpoint değişikliği."

"Anahtar OpenShift'te koda nasıl görünüyor?"

"Anahtar koda verilmiyor, **pod'a** veriliyor — Secret'tan ortam değişkenine ya da IRSA ile. Kod launcher'ın alt süreci olduğu için ortamı devralıyor. Ortam temizlense bile /proc/1/environ aynı UID'den okunabiliyor. **Aynı container'da sır saklanamaz** — mesele dikkatsizlik değil, süreç modeli."

"artifact_ara neden gerekli?"

"Model beyanı yazarken sandbox henüz yok — keşif host tarafında olmak zorunda. Manifest de kesik: depoda 102 satır varken 41'i görünüyor, 176 farklı addan 33'ü hiç ulaşamıyor. Canlı örnek: model listede 4 eşleşme gördü, arama **24** buldu."

"Hubble sandbox'ın erişemediğini kanıtlıyor mu?"

Dikkat — hayır.

"Cilium politikası **pod** düzeyinde, container düzeyinde değil. Hubble'da ptc-run-xxx → artifact-service akışını gördüğünüzde bu **sidecar'ın** trafiği. Sandbox container'ının erişememesinin sebebi ağ politikası değil: anahtar yok, SDK kurulu değil, servis adresi ortamında tanımlı değil."

"Bu tabloda kendinizi birinci yapmışsınız"

"Haklısınız, dairesellik var — ölçütleri de biz seçtik. O yüzden sinyal sıralamasına değil **sinyal/bayt'a** bakın: orada Codex önde. Ve tabloda bizim eksliğimiz açıkça duruyor: tekrar tespiti sütunu."

"Kaç test var?"

"242 test geçiyor. Kabul testi bu cluster'da 52/52. Hata biçimi karşılaştırmasının 17 testi de bulguların kendisini sabitliyor — biçimlendiricilerden biri değişirse tablo sessizce yanlış olmasın diye."

4 · Söyleme listesi

x "bizimki Files API'nin aynısı"	→ soy, alias, arama yok
x "Hubble sandbox'ı hapsediyor"	→ pod düzeyi, container değil
x "KFP güvensiz"	→ kendi bağlamında doğru karar
x "ODF'de de test ettik"	→ ODF kapsam dışıydı, TEST EDİLMEDİ
x "MinIO OpenShift'in çözümü"	→ PoC'nin vekili; üretimde ODF/harici S3
x "en iyi biçim bizimki"	→ en iyi = bu ölçütlerde; düzeltme başarısını ÖLÇMEDİK

MinIO lisansı (AGPLv3) sorulursa: "kurumsal kullanımda hukuk tarafına sorulması gereken bir konu, bu projede doğrulamadım."

5 · Zaman kısılarsa

15 dakikaya sıkışırsan şu 5 sayfa yeter:

1	problem	
3	dört aile	← tez
5	Red Hat	← en yakın komşu
9	iki container	← bizim cevabımız + OpenShift
11	ölçüm	← kanıt

2, 4, 6, 7, 10 destekleyici. 8'i atlarsan beyan mekanizması havada kalır — 9'da bir cümleyle kapat: *"model ne okuyacağını beyan ediyor, sidecar kod başlamadan yerine koyuyor."*

6 · Kapanış cümlesi

"Piyasada dört yerleşim var ve hepsi kodun güvenilir olduğunu varsayıyor. Bizim kodumuzu LLM yazıyor. O yüzden taşıyıcıyı kodun yanından çıkarıp ayrı bir container'a koyduk — Argo'nun yaptığı gibi. Üstüne pipeline dünyasından üç şey aldık: **beyan, soy, alias**. Hepsi OpenShift'te varsayılanlarla çalışıyor."

7 · Sunumdan sonra — kapatma

```
ss -ltnp | grep -E ':(8123|8080|4245|12000)' # pid'leri gör  
kill <pid>
```

pkill -f kullanma — kendi komut satırıyla eşleşip kabuğu öldürüyor.

8 · Kısa konuşma metni — sayfa sayfa

Baştan sona okunabilir hâli. Her blok **o sayfanın iskeleti**: sayfa süresinin yaklaşık yarısını doldurur, kalanını slayta bakıp doğaçlarsın. §1 *nasıl* anlatacağını söylüyor; burası *ne söyleyeceğini*.

Sunum sırasında yalnızca bu bölüm açık dursa yeter.

Sayfa 1 • Sandbox ölür. Ürettiği kalır.

Kod çalıştıran her sistemin aynı çelişkisi var. Sandbox yaşarsa bir çalıştırma diğerine bulaşır — izolasyon yok. Sandbox ölürse kırk saniyelik iş her turda baştan yapılır. Çözüm ikisinin ortası değil: sandbox ölecek, ürettiği kalacak. Ama bizde bir şart var — bu sandbox'ın içindeki kodu insan değil, **LLM yazıyor**. Yani güvenilmez. Sunumun geri kalanı tek bir varsayıma dayanıyor, o da bu. Bu problemi ilk biz görmedik; önce piyasanın nasıl çözdüğüne bakalım.

Sayfa 2 • Herkes aynı beş soruyu cevaplamak zorunda

On ürüne baktım. Biri sandbox satıyor, biri pipeline, biri ajan framework'ü — hepsi farklı bir problem çözdüğünü sanıyor. Ama hepsi aynı beş soruyu cevaplamak zorunda kalmış: kod **nerede** koşuyor, ne kadar **yaşıyor**, veri nasıl **girip çıkıyor**, depo **anahtarı** nerede duruyor, ve künye **tutuluyor mu**. Slaytta dördü sütun; nerede çalıştığı ile veriyi nasıl taşıdığı aynı kararın iki yüzü olduğu için onları "tezi" sütununa katladım.

Dört satırı konuşayım. **Anthropic** sorunu kalıcılık değil, ortamı yaşatmak olarak görüyor: container otuz gün duruyor, beş dakika hareketsizlikte donduruluyor. Defteri var — ama parantez içine bakın, soy yok. `file_abc123` size dosyayı verir, neyden türediğini söylemez.

Cloudflare'in tezi artifact'le ilgili bile değil: modeller kod yazmayı tool çağdırmaktan daha iyi biliyor. R2 bucket'ını sandbox'a mount ediyor. Defter hücrelerine dikkat — orada "yok" yazmıyor, "**olamaz**" yazıyor. Çünkü mount'ta araya girecek yer yok; kimse durmuyorsa künye de tutulamaz.

Red Hat'in satırı tablonun en ilginç. Defteri en zengin olan ürün: MLMD — tip, soy, beyan edilen girdiler. Ama anahtar sütununda büyük harflerle EVET yazıyor. En iyi defteri tutan ürün, aynı zamanda anahtar kodun yanına koyan ürün. Bu bir çelişki değil, bir varsayım — beşinci sayfada ne olduğunu göreceğiz.

Bir de **Databricks**: mount ediyor ama defteri var. Aracıyı kaldırmamışlar, sürücünün içine gömmüşler. Kalanlar iki gruba ayrılıyor — altyapı satanlar "depoyu sen getir" deyip defter tutmuyor, Devin ise sandbox'ı hiç öldürmeyerek soruyu ortadan kaldırıyor.

Şimdi tek bir sütuna bakın. Defteri olanlar: Anthropic, Google, Red Hat, Databricks. Olmayanlar: Cloudflare, AWS, E2B, Modal, Vercel. İkinci grubun ortak özelliği ne? Hepsi mount ediyor. Araştırmanın en keskin bulgusu şu: **kayıt defteri yalnızca yazma yolu bir bileşenden geçtiğinde ayakta kalıyor**. Bu bir tercih meselesi değil — mount edince künye tutmak imkânsız hâle geliyor. Peki neden? Cevap, baytı kimin taşıdığına.

Sayfa 3 • Baytı kim taşıyor — dört aile

Dört yerleşim var. **A**'da kod doğrudan bucket'a yazıyor; FUSE bunu dosya gibi gösteriyor ama altında HTTP var, araya girecek yer yok — az önceki "olamaz" hücresinin sebebi bu. **B**'de bir taşıyıcı var, ama kullanıcı koduyla aynı container'da: var ama atlanabilir. **C**'de taşıyıcı ayrı bir güven alanında, kod ona ulaşamıyor — bizim yerleşimimiz bu. **D** tek istisna: mount var ama sürücü her erişimde izin soruyor.

Bunu biz düşünmedik, Argo deneyerek buldu. Dört farklı yerleşim denemişler — docker, kubelet, k8sapi, pns — ve v3.4'te hepsini kaldırmışlar. Docker için gerekçeleri tek cümle: *breaks security completely*. Şimdi bu dört aileyi gerçek ürünlerde görelim.

Sayfa 4 • Dört ürün, dört akış

Üstteki ikisinde platform araya giriyor. Anthropic'te kod `$OUTPUT_DIR`'a yazıyor; iş bitince **platform o dizine bakıyor**, dosyayı Files API'ye koyuyor ve konuşmaya bir fiş dönüyor. Container ölse bile bayt kalıyor. Alttaki

ikisinde giren yok: Cloudflare mount ediyor, bakacak kimse yok; ADK ise baytı hiç taşıyor — sadece isimleri her turda talimatlara yazıyor, içeriği model isteyince ve yalnızca o isteğe ekliyor.

Ömürlere bakın: Anthropic otuz gün, OpenAI yirmi dakika. İkisi de keşif sorununu **sandbox'ı yaşatarak** çözüyor. Bizde o kapı kapalı. Şimdi pipeline tarafına, bize en yakın olana bakalım.

Sayfa 5 • Red Hat / OpenShift AI

Bir hat adımı bir pod, içinde iki program. Init container'da `kfp-driver` girdilerin adresini MLMD'den çözüyor. Main container'da `kfp-launcher` PID 1 olarak duruyor: önce dosyaları indiriyor, sonra kullanıcı kodunu çalıştırıyor, sonra çıktıları yükleyip künyeyi yazıyor.

Kullanıcının kodu S3'ü hiç görmüyor — `cikti.path`'e yazıyor, yerel bir yola. İndirme kod başlamadan bitiyor, yükleme kod bittikten sonra başlıyor.

Ama launcher kullanıcı koduyla **aynı container'da** ve kodu **alt süreç** olarak çalıştırıyor. Alt süreç ebeveyninin ortamını devralır: aynı env, aynı dosya sistemi, aynı ağ. Yani kod launcher'ı atlayıp doğrudan S3'e konuşabilir. Ortam temizlense bile aynı UID `/proc/1/environ`'u okuyabiliyor — aynı container'da sır saklanamaz, bu bir dikkatsizlik değil, süreç modelinin sonucu.

Ve hemen şunu eklemek lazım: **KFP için bu doğru karar**. Oradaki kodu bir insan yazdı, gözden geçirdi, git'te duruyor. Sarmalayıcının işi güvenlik değil kolaylık. Bizde kodu LLM yazıyor; aynı karar savunulamaz.

Sayfa 6 • Tekton • Argo • Databricks • Devin

Argo bizim yerleşimimizin kaynağı: `argoexec` kullanıcının imajından ayrı bir imaj, anahtar orada, kod ona erişemiyor. Tekton bambaşka bir şey yapıyor — artifact'i kap değil **referans** sayıyor; kendi ifadeleriyle *"only metadata and attestations are stored, not artifact content."* Databricks mount ailesindeki tek istisna. Devin ise soruyu ortadan kaldırıyor: sandbox'ı hiç öldürmüyor, microVM snapshot'la RAM'i bile saklıyor — bedeli süresiz yaşayan bir makine. Bizimki dört saniye yaşıyor, ikisi de bize kapalı.

Buraya kadar baytı **taşımayı** konuştuk. Bir sorun daha var: onu **bulmak**.

Sayfa 7 • Keşif ve kayıt defteri

Tek bir şey belirliyor: sandbox yaşıyor mu. Yaşıyorsa `ls` yeter, keşif bedava — otuz gün, yirmi dakika, mount, fark etmez. Pipeline dünyasında ise soru hiç doğmuyor: DAG'ı insan yazıyor, girdi bağlanmış geliyor, seçim yapan bir ajan yok. Bizde iki şart birden var — ajan karar veriyor **ve** sandbox ölüyor. Bu ikisinin bir arada olduğu başka bir yer bulamadım.

Sağda ikinci soru: aynı addan on iki sürüm birikince ne oluyor. MLflow alias koyuyor, KFP'de çakışma imkânsız çünkü yol run-id içeriyor, DVC içerik hash'i kullanıyor. Biz alias'ı MLflow'dan aldık. Piyasa bitti; şimdi biz ne yaptık.

Sayfa 8 • Çağrı değil. Beyan.

Eskiden kod çalışırken `load_artifact` diye bir ağ çağrısı yapıyordu. Şimdi sadece **beyan** ediyor: `inputs` listesine adı yazıyor. Dosya kod başlamadan yerine konuyor, kod düz `open` yazıyor. Üç biçim var: düz ad kendi çıktısını, `workflow/ad` başka bir çalıştırmanınkini, `ad@alias` ise sabitlenmiş bir sürümü getiriyor.

KFP ve Argo tam böyle yapıyor — tek fark, onlarda beyanı insan yazıyor, YAML'da; bizde model yazıyor.

Bir ölçüm: yüz dosya ürettik, kod bir tanesini okudu. Beyan olmadan yüz dosya iniyor ve soy **yüz ebeveyn** oluyor. Beyanla iki dosya iniyor, soy iki ebeveyn. Asıl bedel indirme değil, **soy şişmesi** — yüz ebeveynli bir artifact'in soyu hiçbir şey anlatmıyor. Soyu da beyandan çıkarıyoruz, MLMD'nin `DECLARED_INPUT` olayı gibi. Peki bu nerede çalışıyor?

Sayfa 9 • İki container, bir /output

Pod'da iki container var. Sidecar'da S3 anahtarı ve kapsam jetonu; sandbox'ta LLM'in kodu ve hiçbir sır. Aralarında sadece /output paylaşıyor. Container'lar ortam paylaşmıyor — beşinci sayfadaki alt süreç problemi burada yok.

Ölçtük: sandbox'ın ortamında S3 kimlik bilgisi yok, S3 SDK kurulu değil, MinIO'ya IP ile gidince zaman aşımı, internete bağlantı hatası. Sandbox'ın ağ çağrısı sıfır.

OpenShift tarafı: restricted-v2 SCC ile tam akışı koşturduk — rastgele UID, root olmayan kullanıcı, tüm capability'ler düşürülmüş. SCC'nin dayattığı hiçbir kısıt bizi kısıtlamıyor. Kalıcılık için gereken tek şey bir S3-uyumlu depo: endpoint, bucket, anahtar. Kod hem ObjectBucketClaim hem OpenShift AI connection sözleşmesini okuyor, yani geçiş bir yapılandırma değişikliği — kod değişikliği değil.

Buraya kadar artifact'i konuştuk. Bir eksen daha var: kod patladığında ne oluyor.

Sayfa 10 • Kod patlıyor. Modele ne dönüyor?

Aynı hatayı altı ürüne sorduk. Üçünde aynı üçlü var: çıkış kodu, tam traceback, ve hata anında **korunan stdout** — yani kodun oraya kadar ne yaptığı. smolagents bilinçli bir takas yapıyor: traceback yerine patlayan satırın kaynak metnini veriyor. Bizim eski hâlimiz üçünü de kaybediyordu; üstelik metin modele *dur* diyordu — "tahmini bir değer üretme". Bunu iddia olarak bırakmadık, ölçtük.

Sayfa 11 • Aynı hata, on biçim — ölçülmüş

On biçimlendiriciyi kurup aynı beş arızaya soktuk, yedi ölçütte puanladık. Kendi biçimimiz gerçek kod — `entrypoint.py` içe aktarılıp çağrılıyor, kopya değil. Diğer dokuzu belgelenmiş davranıştan yeniden kuruldu.

Üç bulgu çıktı. Birincisi: **kırpmanın yönü hatayı yiyor**. SWE-agent'ın varsayılan şablonu baştan kesiyor; hata en sonda olduğu için yüz altmış kilobaytlık çıktıda `ValueError` modele hiç ulaşmıyor. Aynı ürünün `bash_only` yapılandırması baş yarısı artı son yarısını alıyor ve kaybetmiyor — yirmi üçten yirmi altıya. Tek satırlık bir ayar farkı.

İkincisi: **kırpmamanın bedeli**. Tek bir hatada AutoGen iki yüz on bir bin, Anthropic iki yüz on beş bin bayt gönderiyor. Bizimki yirmi bin, Codex on bin.

Üçüncüsü: **talimat sütunu neredeyse boş**. On biçimden dokuzu ne olduğunu anlatıyor, ne yapılacağını söylemiyor.

Ve bir dürüstlük notu: ölçüt listesi bizim araştırmamızdan çıktı, bizim biçimimizi de o araştırma şekillendirdi — sıralamada dairesellik var. Kıyaslanabilir olan sinyal değil, sinyal bölü bayt; orada Codex önde. Bizde eksik olan da tabloda duruyor: tekrar tespiti, o sadece OpenHands'te.

Kapanış

Piyasada dört yerleşim var ve hepsi kodun güvenilir olduğunu varsayıyor. Bizim kodumuzu LLM yazıyor. O yüzden taşıyıcıyı kodun yanından çıkarıp ayrı bir container'a koyduk — Argo'nun yaptığı gibi. Üstüne pipeline dünyasından üç şey aldık: beyan, soy, alias. Hepsi OpenShift'te varsayılanlarla çalışıyor.